THE INSIGHT

A price vector is a proof.

For any prices ν, no route through the owner's venues can return more than

$$B(\nu) = \nu_s A + \sum_{\text{venues}} \left[\left(\sqrt{(\nu_b R_b)} - \sqrt{(\nu_a R_a / \gamma)}\right)_+^2 + \left(\sqrt{(\nu_a R_a)} - \sqrt{(\nu_b R_b / \gamma)}\right)_+^2 \right]$$

A is the input, R_a, R_b a venue's live reserves, $1 - \gamma$ its fee. Each term is the most that venue can add at prices ν, in closed form.

Unforgeable

Weak duality: every ν gives a valid upper bound. A dishonest agent cannot fake a proof; it can only fail to have one.

Tight

Strong duality: at the right ν the bound meets the optimum. Our solver finds it in a median 3.3 ms.

Exact on-chain

Every rounding step goes up, so the theorem survives integer arithmetic. The check costs O(venues).

THE MECHANISM · QEDEXECUTION.TRADE

The agent routes freely. The contract checks one inequality.

1 · Order

The co-signer's certificate commits to hash(order): recipient in the owner's mandate, its own floor, τ, the venue set.

2 · Bound

$\hat{B}(v)$ from the live reserves of every owner venue, not only those used, every step rounded up.

3 · Funding

The unchanged QEDVault pays exactly A: mandate, co-signature, caps and solvency all still apply.

4 · Route

Up to 32 hops of any shape: splits, multi-hop, the same pool twice.

5 · Nothing left

No intermediate token may stay behind; unspent input goes back to the vault.

6 · Two checks

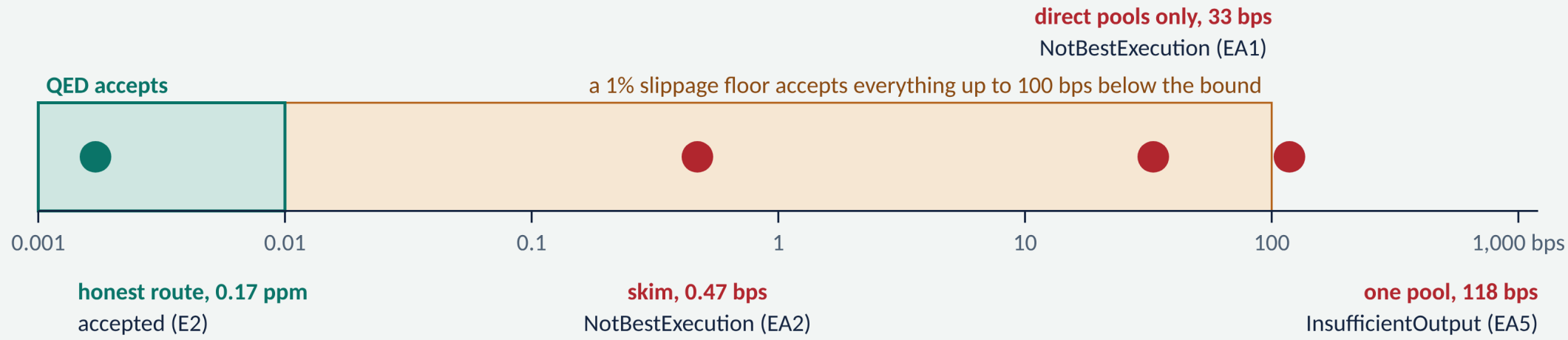
out ≥ floor (market risk) and $\text{out}\cdot 10^6 \geq \hat{B}\cdot(10^6 - \tau)$ (best execution), out measured as a balance change.

Every accepted trade delivered at least $(1 - \tau)\cdot\text{OPT}$ on the state it executed against, whatever route the agent chose.

Proof: docs/ALGORITHMS.md §12 · contract: contracts/src/QEDExecution.sol

A COMPROMISED AGENT, ONE CO-SIGNED ORDER

Every leak a slippage limit allows, refused on-chain



"Forged" prices to bless a bad route → **NotBestExecution** (EA3)

Strand the bridge token → **Leftover** (EA4)

Redirect the output to Mallory → **OrderMismatch** (EA6)

Call without the agent key → **NotOperator** (EA7)

Front-run the signed order → **InsufficientOutput** (EA8)

Then the agent re-plans and **declines** to trade into the moved market (A10)

All on Sepolia under one genuine co-signed order; the honest route then went through with the same certificate

TOPIC F · THE FOUR REQUIRED OUTCOMES

Each outcome is a public transaction you can open.

#	Outcome	Evidence on Ethereum Sepolia
O1	A separate AI-agent key acts under an independent policy certificate	`L4 0x56a2334e`… payment · `E1 0x56f798cd`… trade · `PE1 0xaaa0bacd`… and `QE1 0x12739525`… agent-initiated
O2	Paid work ends in a real escrow settlement	`L6 0x4166253f`… 15 tUSD on an optimality proof · `Z3 0xbb619bf0`… 12 tUSD on a zero-knowledge proof
O3	An adversary cannot breach a liability or tamper with signed terms	48 attacks, 48 reverts, 0 tUSD to the attacker; e.g. `EA1 0xccd07d17`…, and `P3 0x9f4a753c`…: the creditor paid during a pause
O4	A reviewer can reproduce everything	`npm run verify`: every claim re-derived from a public RPC, no keys, in CI daily; 24 contracts verified on Sourcify

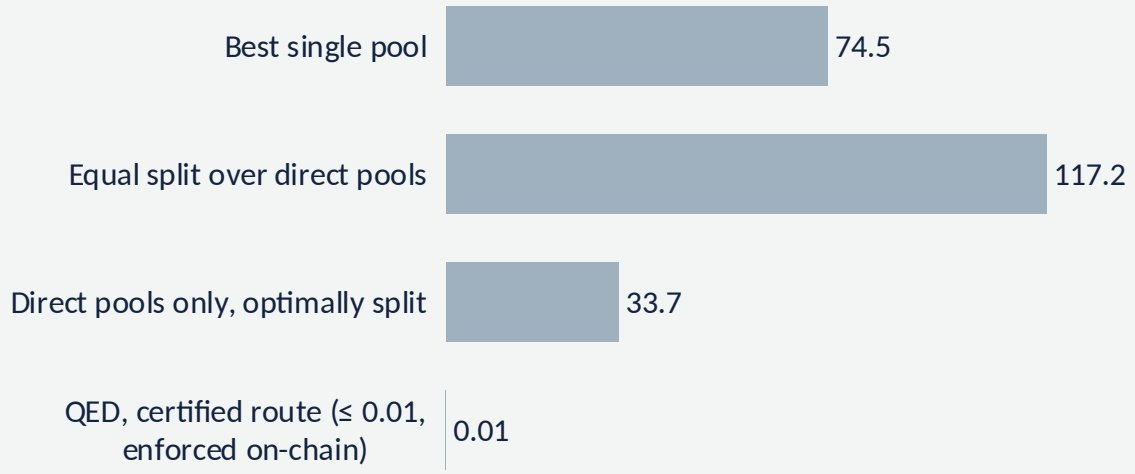
Built on QED's earlier proofs, unchanged: owner mandate × co-signer · ring-fenced solvency · paid work on a KKT proof · bounded loss · proven clearing · zero-knowledge optimality

All 196 transactions with blocks and errors: docs/EVIDENCE.md · backup slides B1–B8

EVALUATION · 1,000 SEEDED MARKETS

What the proof buys, including where it buys little

Median shortfall below the optimum, bps



14–32 bps

median a hostile agent keeps per trade while still clearing a 1% floor (max 99.9). With QED: ≤ 0.01.

0.075 ppm

median gap between our route and its own bound; 92.3% within 1 ppm; solver 3.3 ms median.

3.2 bps

Honest limit: for trades under 0.5% of depth a good router is already this close. There the proof buys the guarantee, not the average.

benchmarks/run.ts · fixed seeds, re-run and diffed in CI · baselines are simple policies, not production aggregators

FALSIFIABLE, FOR MONEY

Beat the bound and the contract pays you 1,000 tUSD.

EXECUTIONBOUNTY · 0X3148…19C4

Runs your route on its own funds and compares the output with the deployed QEDExecution's bound. No judge, no owner, no withdrawal.

XA1	the optimal route itself → TheoremHolds
XA2	prices lowered 3% to shrink the bound → TheoremHolds
XA3	the whole 1,000 tUSD in one trade → TheoremHolds

3,000 tUSD

unclaimed across three theorems: best execution, portfolio optimality, clearing maximality. Checked daily in CI.

6,000

fuzzed routes, prices and claims never beat the bound; weak duality also holds on 12,000 route–price pairs off-chain.

Positive control

against a bound understated by 0.1%, the bounty pays the first real refutation, so the check is not vacuous.

145 Solidity + 60 TypeScript tests · contracts/test/QEDExecution.t.sol

CHECK US · AND OUR LIMITS

Don't trust the slides. Re-derive them.

`npm run verify`

every claim from a public Sepolia node, no keys: bytecode, receipts, replayed reverts, each bound recomputed, routes replayed

`QED_OFFLINE=1 npm run local:execute`

the whole best-execution economy and the policy loop with dev keys; PE1 and PE2 reproduce to the raw unit

`forge test · npm run test:ts`

145 + 60 tests; 24 contracts verified on Sourcify; all of it in CI on every push

What QED does not do

The proof is about the state a trade executes on: a front-run smaller than the 1% co-signed floor is not caught.

Venues must be honest constant-product pools the owner names; concentrated liquidity is not covered yet.

The policy contract enforces safety, not liveness: a slow or idle agent is not caught.

Testnet, valueless tokens, not audited.

PROOF 1 · AUTHORITY

The AI’s key can pay only under two signatures it doesn’t hold.

The owner’s mandate

Signed once, off-chain, with a cold key. Names the payees, the caps and an expiry.

The co-signer’s certificate

A separate process with its own key runs 12 checks, then signs the exact recipient, amount, date, a single-use id and the hash of its decision record.

The vault’s own rules

Payee in the mandate, caps, no replay, solvency. Change one signed byte and the payment reverts.

ON SEPOLIA · TRANSACTION L4

`0x56a2334e…83b6`

Sent by	the AI agent’s key 0x2A10…d0Bb
Mandate signer	the owner 0xb7Ed…97f7
Certificate signer	the co-signer 0xf1F8…2402
Evidence hash	0xb0dd4c99… = keccak256 of the published record, 12 of 12 checks
Paid	6 tUSD to Helix Data for a licensed risk model



PROOF 2 · SOLVENCY

Money owed to a creditor can only ever reach that creditor.

70 tUSD ring-fenced for CloudCo’s compute bill	30 tUSD free

$$B \geq P$$

Balance covers every ring-fenced liability at every block. Proved by induction; fuzzed with both hot keys stolen.

Who can move it?

Not the AI. Not the co-signer. Not even the owner. Only settlement to the creditor, or the creditor releasing its own claim.

Pause stops the AI, not the creditors

On Sepolia the vault was paused, then the attacker’s own address called settle(). CloudCo received exactly 70 tUSD (P3).

QED before the upgrade, unchanged and still live

PROOF 3 · CORRECTNESS

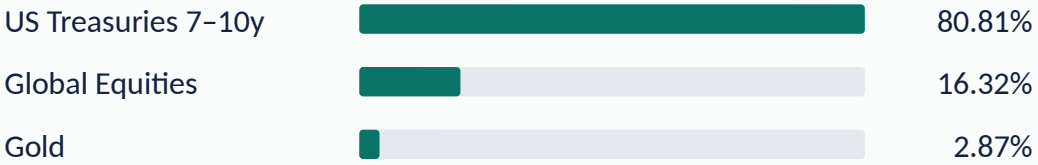
The escrow pays for a proof, not a promise.

$$\text{minimise } \tfrac{1}{2}\, w^{T}\Sigma w \text{ s.t. } \Sigma w_i = 1,\ w \geq 0$$

The AI solves it off-chain and submits the portfolio with its multipliers and a factor L. The contract checks, in exact integers:

Condition	What it rules out
w ≥ 0, weights sum to 1	short selling, leverage
μ ≥ 0	fake multipliers
Σw = ν + μ, within τ	any portfolio that isn't stationary
Σ $w_i\mu_i$ ≤ τ	a duality gap: suboptimal work
Σ = LL^{T}, within τ	a non-convex trick

DELIVERED AND VERIFIED ON SEPOLIA · L6



Bitcoin, IG credit, REITs: 0%, each with μ > 0, which proves adding them would raise risk.

$$f(w) - f^{*} \leq (2n + 3 + |\nu|)\cdot\tau \approx 1.5 \times 10^{-8}$$

Residual 0 · duality gap 0 · about 0.3 M gas to check

QED before the upgrade, unchanged and still live

OUTCOME 2 · PAID WORK, SETTLED BY PROOF

One business cycle, end to end, on Sepolia

1
Request
A robo-advisor agent asks for an optimal portfolio, budget 20 tUSD

2
Quote
QED prices it at 15 (cost 6) and signs the terms

3
Escrow
The client locks 15 tUSD with the signed quote (L2)

4
Buy data
The AI key pays 6 tUSD for the risk model, co-signed (L4)

5
Solve
1.7 ms to the exact optimum and its proof

6
Settle
The chain checks the proof; 15 tUSD lands in the vault (L6)

Meanwhile, a cheaper rival bid 9 tUSD and tried to cut corners

Lazy 1/N portfolio → **DualityGapTooLarge**, gap 0.0191 (X17)

Optimum nudged 0.01% → **StationarityViolated** (X18)

Deadline passed → the client got all 9 tUSD back (X19)

Result: +9 tUSD margin for QED, the ring-fence untouched, the dishonest provider paid nothing.

OUTCOME 3 · ADVERSARIES

19 attacks on the public chain. 19 reverts. 0 tUSD to the attacker.

Tx	Attempt	Reverted with
X2	Edit price in signed quote	BadProviderSignature
X3	Redirect quote's payee	BadProviderSignature
X5	Stolen AI key, forged co-sign	BadCosignerSignature
X6	Both hot keys pay attacker	PayeeNotInMandate
X7	Self-signed mandate	BadOwnerSignature
X8	Real cert, amount 6 → 9	BadCosignerSignature
X9	Real cert, new recipient	BadCosignerSignature
X10	Replay a real certificate	IntentAlreadyUsed
X11	Both keys overdraw free funds	SolvencyViolation
X12	Both keys exceed the 24 h cap	ExceedsDailyCap
X13	AI key calls withdraw()	NotOwner
X14	AI key voids creditor's claim	NotCreditor
X15	AI key lifts its own cap	NotOwner
X17	Lazy 1/N portfolio	DualityGapTooLarge
X18	Optimum nudged 0.01%	StationarityViolated
H4	Published keys pay attacker	PayeeNotInMandate
H5	Published keys take 50	ExceedsPerPaymentCap
H6	Published keys withdraw()	NotOwner
P2	Valid payment while paused	VaultPaused

Run 1 only; 29 more in runs 2–4 (48 in all): docs/EVIDENCE.md

PROOF 4 · BOUNDED LOSS

We published the AI's keys. The money stayed put.

WORST CASE IF BOTH HOT KEYS ARE STOLEN

$$\mathrm{BR}(h) = \min(\ \text{free},\ \text{cap left} + k{\cdot}D\ )$$

Zero while paused. Computed on-chain by QEDLens.blastRadius() at any block, and fuzzed as an upper bound.

Loss to any address the owner didn't approve: exactly 0.

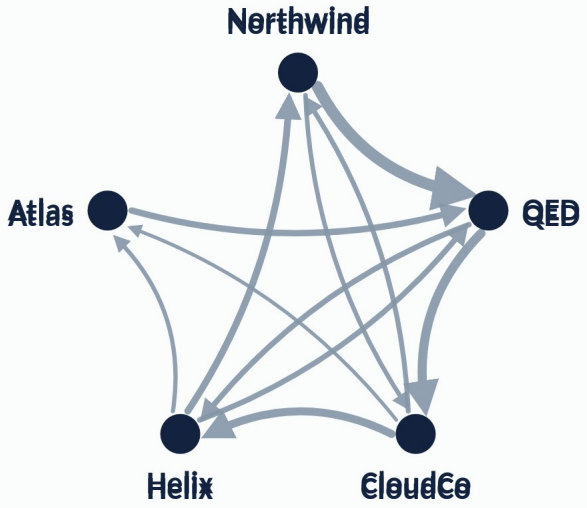
HONEYPOT VAULT 0XE5B3…7F04

Published	its AI key and co-signer key, in our repo
Ring-fenced	90 tUSD for CloudCo until 30 Jun 2027
The keys can	pay CloudCo up to 1 tUSD a day (H3)
The keys can't	pay anyone else (H4), take more (H5) or withdraw (H6)
Check it	live on the dashboard, any block

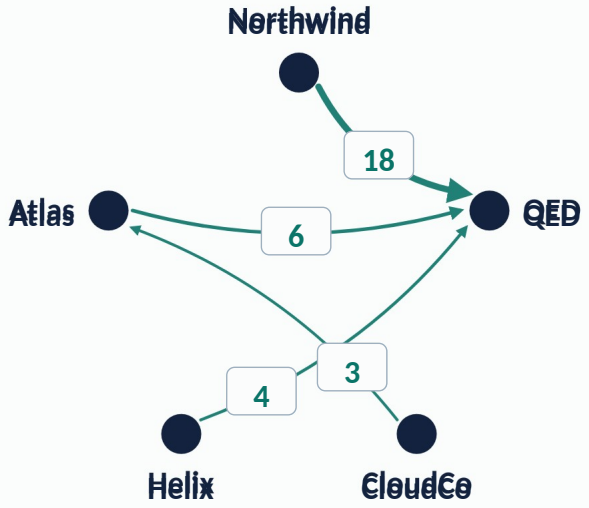
RUN 2 · CERTIFIED CLEARING

A gridlocked agent economy, cleared by proof

Before · 200 tUSD in 11 debts



After the proven netting · 31 in 4



Gross debt 11 debts, 5 AI businesses	200
Cash they hold paying gross reverts (G1)	28
Naive netting NotProvablyMaximal (N1)	162
Proven netting certificate holds (N6)	**169**
Paid in tokens net debits only (S1–S4)	31

$$\max \Sigma x \text{ (net positions fixed)} = \min \Sigma d{\cdot}y \text{ s.t. } \pi_u - \pi_v + y \geq 1$$

exact integers · 6 more attacks refused

QED before the upgrade, unchanged and still live

RUN 3 · ZERO-KNOWLEDGE OPTIMALITY

Paid for a proof about data nobody saw

ON-CHAIN, PUBLIC	NEVER ON-CHAIN
Data commitment, signed by Helix `0x0ec5…6983`	**Covariance model, 21 entries**
Portfolio commitment `0x0577…ba68`	**Portfolio weights, 6 sleeves**
Certified variance `0.0015702689`	**KKT multipliers v, μ**
Proof bound to `QED agent`	**Convexity witness L**

Groth16 statement 12,569
the full KKT certificate

Proved in 1.5 s
by the QED agent, one CPU

Paid by job #1's escrow **12 tUSD**
unchanged contract (Z3)

Refused 3 / 3
copied proof, lower variance, self-attested data (ZA1–ZA3)

Accepted ⇒ $\frac{1}{2}w^{\mathrm{T}}\Sigma w \le \mathrm{OPT} + 1.5 \times 10^{-9}$, for the model the vendor signed, and the chain learns nothing else.

AUTONOMY · SEPOLIA RUNS 4C AND 4D

One signed policy. The agent decided when and how much.

STANDING POLICY · SIGNED ONCE

Keep at least **45 tEUR** free in the euro reserve. Tranches of 2–25 tUSD. At most **150 bps** execution cost each.

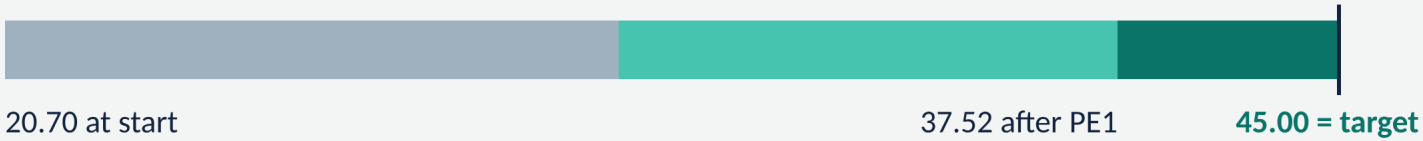
Each tick the agent converts

$$X = \min(\text{need}, \text{impact}, \text{policy cap}, \text{mandate cap}, \text{free funds})$$

and waits if X < 2 tUSD. The co-signer re-derives X before it signs; since Run 4d a contract refuses any tranche the policy does not need.

Tick 0	An over-eager agent asks for 25 tUSD at once	refused: co-signer allows 19.54, 180 bps > 150 (PA1)
Tick 1	Short 24.30 tEUR → convert 19.54 tUSD, limited by **market impact**	PE1 · 16.816098 tEUR · 0.18 ppm
Tick 2	Short 7.48 tEUR → convert 8.84 tUSD, limited by **need**	PE2 · 7.484336 tEUR · 0.40 ppm
Tick 3	Reserve free: 20.70 → 37.52 → **45.00 tEUR**. Target met.	nothing sent

Free tEUR in the euro reserve



Runs with dev keys too: npm run local:execute reproduces PE1 and PE2 to the raw unit

THE PROBLEM

An agent that converts money can leak a little on every trade, and nothing checks it.

A floor, not a proof

Wallets and intents accept any route above a slippage limit. The agent keeps the difference.

Routes that clear a 1% floor still give up a median 14–32 bps (1,000 markets).

Checked afterwards

Solver auctions compare fills with baseline pools after settlement and punish by vote.

CoW's best-price rule is “not enforced by the smart contract”.

Unseen discretion

Whether, when and how much to convert is the agent's call. Today a human approves each trade, or nobody does.

Brokers owe clients **best execution** by law (MiFID II art. 27, FINRA 5310). An AI agent owes it to no one.